Manual del Programador — Modelo de Datos con `struct` y Punteros

Campo	Detalle
Proyecto	Sistema Proyecto — Modelado de datos con estructuras y punteros
Actividad	5 — Modelado de Datos con Estructuras (struct) y Punteros
Versión	v1.0.0
Alumno	Alexis Maier — Grupo 4
Institución	E.E.S.T. N° 1 "Eduardo Ader" — Vicente López
Materia	Laboratorio de Programación (LPR) — 5° Año 3° División — Grupos A-B
Profesores	Mansilla Muñoz York Elías / Ganduglia Yamil
Archivo fuente	`src/main.cpp`

1. Descripción general

El programa modela una entidad de proyecto mediante una estructura de datos (`struct`) y la carga en memoria usando **pasaje por dirección** (punteros). Sirve como base reutilizable para el Proyecto Integrador Anual, especialmente para representar lecturas de sensores, estado de módulos de hardware o registros de stock.

El módulo expone una única función pública:

```
void cargarDatos(EntidadProyecto* ptr);
```

que recibe la **dirección** de la estructura y escribe directamente en sus campos.

2. Modelo de datos

```
struct EntidadProyecto {
int id; // Identificador numérico de la entidad
char nombre[50];// Nombre o descripción en texto
float metrica; // Valor decimal de operación (sensores, consumo, avance)
};
```

Campo	Tipo	Tamaño	Descripción
`id`	`int`	4 bytes	Identificador de la entidad (por ejemplo, el número del sensor)
`nombre`	`char[50]`	50 bytes	Nombre o descripción legible
`metrica`	`float`	4 bytes	Lectura numérica decimal

Tamaño total: 58 bytes útiles, ajustados a **60 bytes** por la alineación de memoria que exige el procesador (los campos de 4 bytes deben caer en direcciones múltiplos de 4). Por esa razón el arreglo de caracteres se ubica en el medio de la estructura: colocarlo al final agregaría 2 bytes de relleno adicionales.

3. Inicialización de seguridad

```
EntidadProyecto miEntidad = {0, "Sin cargar - LPR 5to", 0.0f};
```

Inicializar la instancia antes de usarla evita que contenga **valores residuales** de la memoria (*basura*), garantiza un comportamiento coherente ante un error previo y documenta el estado inicial del objeto.

4. Carga de datos por dirección

```
cargarDatos(&miEntidad); // en main()
```

El operador `&` (dirección de) entrega la dirección física de la estructura. Dentro de la función los campos se acceden con el **operador flecha** `->`:

```
void cargarDatos(EntidadProyecto* ptr) {
cin >> ptr->id;
cin.ignore(); // limpia el Enter residual
cin.getline(ptr->nombre, 50); // lectura de la cadena
cin >> ptr->metrica;
}
```

Regla fundamental

Contexto	Operador	Ejemplo
`Variable de tipo struct`	`punto .`	`miEntidad.id`
`Puntero a struct`	`flecha ->`	`ptr->id`

`ptr->id` es azúcar sintáctico equivalente a `(*ptr).id`: primero se desreferencia el puntero para obtener la estructura y luego se accede al campo. Olvidar el asterisco dentro de los paréntesis produce errores de compilación con mensajes poco intuitivos.

5. Verificación posterior

Al regresar a `main()` se imprimen los campos desde la propia variable, lo que demuestra que el pasaje por dirección surtió efecto:

```
cout << "ID Registrado: " << miEntidad.id << endl;
cout << "Nombre Registrado: " << miEntidad.nombre << endl;
cout << "Metrica Guardada: " << miEntidad.metrica << endl;
cout << "Direccion RAM Hexadecimal: " << &miEntidad << endl;
```

La dirección en formato hexadecimal (`0x...`) permite demostrar que la estructura ocupa una región contigua de memoria.

6. Control del búfer de entrada

`cin.ignore();` es **obligatorio** entre `cin >>` y `cin.getline()`. El lector numérico deja el carácter Enter (`\n`) pendiente en el búfer; sin esa limpieza, `getline()` lo leería como única entrada y devolvería una cadena vacía, salteando el campo `nombre`.

7. Compilación y ejecución

```
g++ src/main.cpp -o src/sistemaproyecto.exe
.\src\sistemaproyecto.exe
```

Requisitos: Visual Studio Code con extensión C/C++ de Microsoft y compilador MinGW-w64 (GCC/G++) agregado al `PATH` del sistema.

8. Errores frecuentes y soluciones

Error / síntoma	Causa	Solución
`no match for 'operator->'`	`Se usó . sobre un puntero`	`Cambiar a ->`
`nombre aparece vacío`	`Falta cin.ignore() antes de getline()`	`Insertar cin.ignore();`
`invalid conversion from 'int*'`	Se pasó el valor en vez de la dirección	`Invocar con &miEntidad`
El valor no cambia tras la función	Se pasó la estructura por valor	`Pasar con & o *`
Caracteres raros en la salida	`Falta chcp 65001 en la consola`	`Ejecutar chcp 65001 antes`

9. Relación con el Proyecto Integrador Anual

La estructura puede adaptarse para representar cualquier entidad del prototipo de Expotécnica: sensores, módulos de hardware, usuarios, turnos o stock. El pasaje por dirección evita copiar estructuras grandes en cada llamada, lo que resulta especialmente relevante al procesar datos de sensores en tiempo real.

10. Referencias

- cppreference.com. (s. f.). *Struct*. Referencia del lenguaje C++. https://en.cppreference.com/w/cpp/language/struct
- cplusplus.com. (s. f.). *Structures*. Tutorial de C++. https://cplusplus.com/doc/tutorial/structures/

- Microsoft Learn. (s. f.). *Operadores de miembro > y . . https://learn.microsoft.com/es-es/cpp/cpp/member-pointer-operators
- Microsoft Learn. (s. f.). *Operador sizeof*. https://learn.microsoft.com/es-es/cpp/cpp/sizeof-operator